Extractive Question Answering on SQuAD 2.0 with a Fine-Tuned DistilBERT

David Falaye · Kosisochukwu Ogbuanya

CSCI 390: Natural Language Processing and Applications · Dr. Qi Li · November 2026

Abstract. We fine-tune DistilBERT for extractive question answering on SQuAD 2.0, where the model must either mark the answer inside a passage or say that the passage has no answer. Our model reaches 64.1 exact match and 67.4 F1 on the development set, above our target of 65 F1, and 68.1 F1 after tuning the no-answer threshold. It is 15.5 F1 points behind a published RoBERTa model at 54% of its size and 1.8 times its speed. An analysis of all 4,258 errors shows that three quarters of them are wrong decisions about *whether* to answer, not about *where* the answer is. As an extension we test the model on whole articles, where it loses 16 to 23 F1 points, and show that a different way of combining text windows recovers 6 to 7 of them.

1. Introduction

Extractive question answering takes a passage and a question and returns the words of the passage that answer the question. SQuAD 2.0 [1] makes the task harder by adding questions that look answerable but are not, so a system also has to know when to say "no answer". This matters in practice: a system that always answers will confidently return something wrong.

Our goal was to build the whole pipeline ourselves, from raw data to a working demo, with a model small enough to train on a free GPU. We chose DistilBERT [2], which has 40% fewer parameters than BERT. We compare it with `deepset/roberta-base-squad2`, a larger model fine-tuned by others on the same data, and we study where our model fails. All code, results and the demo are public (links at the end).

2. Data

SQuAD 2.0 consists of Wikipedia paragraphs with questions written by crowd workers. The training set has 130,319 questions, of which 33.4% are unanswerable. The official test set is hidden, so we use the development set as our test set: 11,873 questions over 35 articles, half of them (50.1%) unanswerable. We never used it for training. Each answerable development question has several reference answers; a prediction is scored against the closest one.

Passages are short (median 142 tokens, 90th percentile 236) and so are answers (median two words). Only 0.9% of the examples are too long to fit in one 384-token input. Questions starting with "what" make up 63% of the development set, and "why" questions only 1.6%.

3. Method

Preprocessing. The question and the passage are tokenized together as one sequence of at most 384 tokens. A longer passage is cut into overlapping windows that share 128 tokens, so an answer near a cut is whole in at least one window. SQuAD gives each answer as a character position, while the model predicts token positions, so we convert between the two with the tokenizer's offset mapping. Unanswerable questions, and windows that do not contain the whole answer, are labelled with position 0, the `[CLS]` token. To check this step we decoded the labels of 2,000 training examples back into text: 99.7% matched the reference answer exactly and none were lost.

Model and training. DistilBERT gets one new linear layer that gives every token a start score and an end score. We fine-tuned all weights for 2 epochs (16,470 steps) with batch size 16, learning rate 3e-5 with linear warm-up over the first 10% of steps, weight decay 0.01 and mixed precision. Training took 44 minutes on one Tesla T4 in Google Colab, and the loss fell from 5.81 to about 0.9. Before the full run we ran a 50-step test on 1% of the data to make sure that the loss fell and that checkpoints were saved.

Post-processing. The score of a span is its start score plus its end score. In each window we try the 20 best start positions with the 20 best end positions and discard spans that fall outside the passage, run backwards, or are longer than 30 tokens. "No answer" has a score too: the start plus end score of `[CLS]`. The model abstains when the no-answer score exceeds the best span's score by more than a threshold τ, which is 0 unless stated otherwise.

Evaluation. We report exact match (EM) and F1, the word overlap between prediction and reference, after the standard normalization. We wrote the metrics ourselves, so that we have a score for every question, and the script stops if its totals differ from the official SQuAD 2.0 metric. As a check on the whole pipeline, it reproduces the published RoBERTa scores (79.9 EM, 82.9 F1).

4. Results

Model	EM	F1	Answerable F1	Unanswerable F1	Parameters	CPU time
DistilBERT before fine-tuning*	1.2	4.6	7.0	2.2	66 M	
DistilBERT (ours)	64.14	67.38	72.17	62.59	66 M	31 ms
RoBERTa (deepset)	79.90	82.91	84.01	81.82	124 M	57 ms

Table 1. Development-set scores at $\tau = 0$. CPU time is the median time to answer one question on an Apple M2 laptop. *First 1,000 questions only.

Table 1 shows that fine-tuning is what makes the model work, and that we met the 65 F1 target. Our model is 15.5 F1 points behind RoBERTa, with 54% of the parameters and 1.8 times the speed. The gap is larger on unanswerable questions (19.2 points) than on answerable ones (11.8).

The no-answer threshold. Because every question's best span and score difference were saved, we could try other thresholds without running the model again (Figure 1). Both models do best with a negative τ, which means abstaining more often. For our model the best value is -1.38, which raises F1 to 68.30. That number is optimistic, because the threshold was chosen on the questions it is scored on. We therefore split the 35 articles into two halves, chose τ on one half and scored the other: the cross-validated F1 is 68.08 (83.44 for RoBERTa). The gain is real but small, and it is a trade: at the best threshold, F1 on answerable questions falls from 72.2 to 66.3 while F1 on unanswerable ones rises from 62.6 to 70.3.

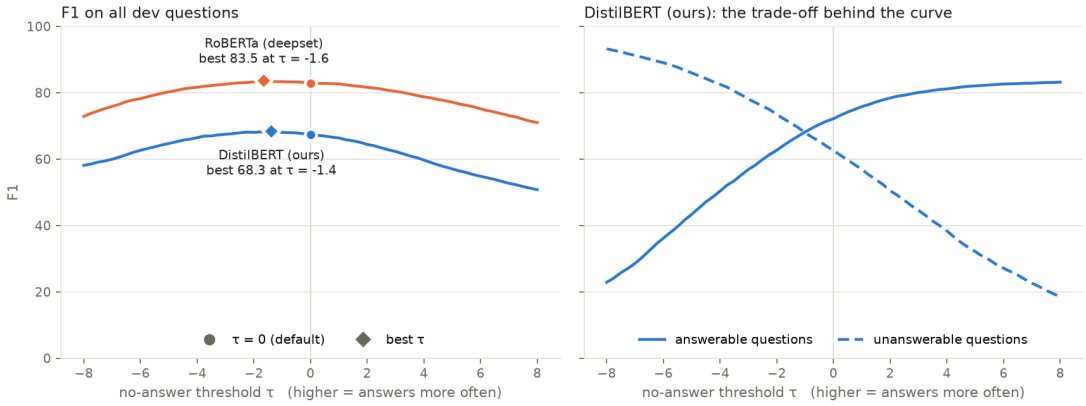


Figure 1. Left: F1 against the no-answer threshold for both models. Right: for our model, a higher threshold helps answerable questions and hurts unanswerable ones.

5. Error analysis

We sorted each of our model's 4,258 wrong predictions into one type automatically (Table 2, Figure 2).

Error type	Ours	Share of our errors	RoBERTa
False answer: answered an unanswerable question	2,224	52.2%	1,081
Missed answer: abstained on an answerable question	1,033	24.3%	572
Too long or too short: right place, different edges	573	13.4%	497
Partial overlap or unrelated span	428	10.1%	236

Table 2. Error types on the development set.

Three quarters of our errors are decisions about whether to answer. Finding the span is the smaller problem, and most of RoBERTa's advantage (1,143 of its 1,872 fewer errors) comes from giving fewer false answers. By question word, our model is strongest where the answer is a number or a date ("how many" 85.0 F1, "when" 84.2) and weakest where it is an explanation ("why" 58.7, "how" 64.2). F1 is steady at 71 to 74 for answers of up to four words and drops to 50.9 for answers of ten words or more. RoBERTa shows the same pattern at a higher level.

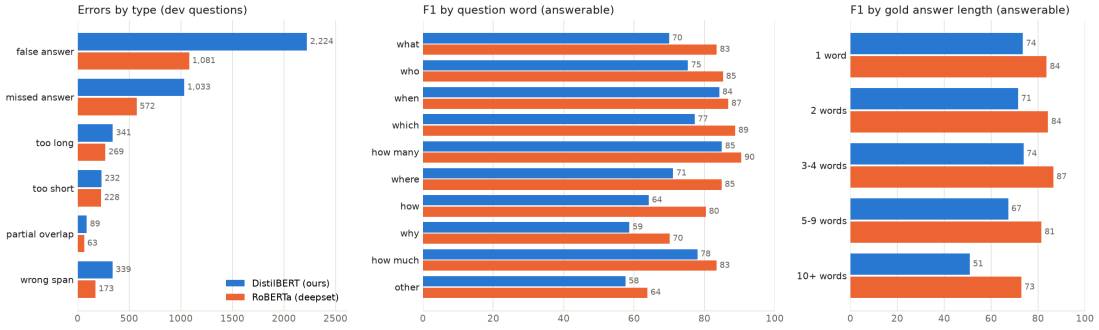


Figure 2. Error types, and F1 on answerable questions by question word and by answer length.

To learn why, we read a random sample of 60 errors covering every type and labelled each by hand.

- **False answers.** Of 27, 24 were questions that reuse the passage's wording and change one thing: 14 swap a detail (a name, a date, the subject of the sentence) and 10 reverse the meaning ("united the tribes" becomes "fought against the tribes"). The model matches the wording and misses the change.
- **Missed answers.** In 10 of 15, the model's best span was the correct answer, but it abstained. These are the cases a higher threshold wins back, at the price of more false answers.
- **Too long or too short.** All 9 were answers a person would accept, such as "Karakorum in Mongolia" for "Karakorum". Exact match counts them as wrong, so it understates how often the model is right.
- **Wrong spans.** 7 of 9 were distractors: a second date or name that fits the same phrase.

Only one of the 60 errors needed real reasoning. On this dataset the model fails mostly by matching surface wording, not because the questions are deep.

6. Extension: long documents

SQuAD hands the model the paragraph that holds the answer. A real user has a whole document. To measure that case, we glued the paragraphs of three development articles back together and asked every question of each article against the whole article, read in overlapping windows (Table 3).

Article	Tokens	Windows per question	Right paragraph given	Whole article, global rule	Whole article, per-window rule
Amazon rainforest (284 questions)	2,941	12	70.6	54.5	62.0
Normans (208 questions)	5,294	22	60.1	37.3	44.0
Oxygen (415 questions)	6,696	28	59.3	41.5	47.7

Table 3. F1 at $\tau = 0$ when the model reads a whole article.

F1 falls by 16 to 23 points. The model still finds the place: when it answers an answerable question, the span lies in the correct paragraph 73% to 94% of the time. What breaks is abstaining. Our post-processing compares the highest span score from any window with the lowest no-answer score from any window. With 12 to 28 windows, extreme values of both turn up by chance, and F1 on unanswerable questions collapses to between 7 and 13.

We therefore added a second rule. Each window is judged on its own, by its no-answer score minus its own best span, and the window that is most confident decides. This gains 6 to 7 F1 points with no re-tuning, and gives identical results on single paragraphs. Lowering the threshold recovers more for both rules, which shows that a thresh-

old tuned on paragraphs does not carry over to long documents. One caveat: a question that is unanswerable from its paragraph may be answerable elsewhere in the article, so the scores on unanswerable questions are a lower bound.

7. Demo

The demo is a Gradio application. The user pastes a passage and asks a question. It shows the answer highlighted in the passage, the five best candidate spans with their scores, a threshold slider, and a switch between our model and RoBERTa. The model runs once per question; moving the slider only re-decides from the stored scores, so the effect of the threshold can be seen immediately. The demo is started on demand, from a Colab notebook or a laptop, and gives a public link.

8. Limitations and future work

- **Extractive only.** The model copies one continuous span. It cannot combine sentences or explain. For a question such as "Why did the museum board stop the chemical treatments?", where the reason is given in an earlier sentence and never stated directly, both our model and RoBERTa return the neighbouring clause instead of abstaining.
- **Too willing to answer.** Half of our errors are false answers. A separate classifier that decides answerability, or training with more unanswerable examples, are the natural next steps.
- **One model size and one dataset.** We did not search hyperparameters, and we tested only on Wikipedia text. A larger model closes most of the gap, as RoBERTa shows, at twice the cost.
- **Long documents.** Reading every window is slow (0.2 to 0.6 seconds per question for these articles) and less accurate. Retrieving the most relevant paragraphs first would address both.

9. Conclusion

A DistilBERT model fine-tuned for 44 minutes on a free GPU reaches 67.4 F1 on SQuAD 2.0, and 68.1 with a tuned threshold, at about half the size and almost twice the speed of a RoBERTa model that scores 82.9. Its main weakness is not locating answers but deciding when there is none, which shows up in the error counts, in the threshold experiments, and most strongly on long documents.

Links

Code and results: https://github.com/Davidfaleye360/AI-Question-Answering-System. Model: https://huggingface.co/Ingsyling/distilbert-squad2. The demo is started from `notebooks/04_demo_colab.ipynb` in the repository.

References

[1] P. Rajpurkar, R. Jia and P. Liang. Know What You Don't Know: Unanswerable Questions for SQuAD. *ACL*, 2018.

[2] V. Sanh, L. Debut, J. Chaumond and T. Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv:1910.01108*, 2019.

[3] Y. Liu et al. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv:1907.11692*, 2019.

[4] Hugging Face. LLM Course, chapter 7: Question answering. https://huggingface.co/learn/llm-course/chapter7/7.